# CSC 4320/6320 – Chapter 3: Concurrency & Synchronization (Slides + Speaker Notes)

## Slide 1

**Slide Content:**
Operating Systems
CSC4320, CSC6320
FALL 2026
Dr. Roya Hosseini
Chapter 3: Concurrency and Synchronization
01
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Chapter 3

---

## Slide 2

**Slide Content:**
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Copyright and Use Notice
These slides were prepared by Dr. Roya Hosseini for
CSC4320, CSC6320: Operating Systems at Georgia State University.
These materials are provided for instructional use only.
Unauthorized copying, posting, sharing, selling, or redistribution of these materials is not permitted without written permission from the instructor.
(c) Dr. Roya Hosseini, Georgia State University, Fall 2026
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 3

**Slide Content:**
CHAPTER 3
Chapter 3 Outline
03
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
3.1
Foundations of Concurrency
3.2
Race Conditions and Critical Sections
3.3
Software Synchronization Solutions
3.4
Hardware Support for Synchronization
3.5
Mutexes and Semaphores
3.6
Monitors and Condition Variables
3.7
Classical Synchronization Problems
3.8
Practical Concurrent Programming **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 4

**Slide Content:**
CHAPTER 3
Learning Objectives
04
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Trace inter-leavings and explain how a shared update can fail.
Evaluate mutual exclusion, progress, and bounded waiting.
Choose and apply locks, semaphores, and condition variables.
Reason about correctness, fairness, and performance in concurrent programs. **Speaker Notes (Under Slide):**

Connection to Chapter 2: scheduling chooses which runnable thread executes. This chapter studies what happens when those executions access shared state. Detailed deadlock characterization and algorithms belong to Chapter 4.

---

## Slide 5

**Slide Content:**
SECTION 3.1
Foundations of Concurrency
Overlapping activities, shared state, and execution order
3.1 / 8
05
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Overlapping activities, shared state, and execution order

---

## Slide 6

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Concurrent versus Parallel Execution
06
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CONCURRENT
PARALLEL
Two or more tasks make progress during overlapping periods.
A single core can alternate among tasks.
Example: a server overlaps one request's I/O wait with another request's work.
Two or more tasks execute at the same instant.
Multiple execution resources run work simultaneously.
Example: two cores process two image regions at once.
A program can be concurrent even when no two instructions execute simultaneously. **Speaker Notes (Under Slide):**

This slide explains concurrent versus parallel execution
Concurrency: in an operating system refers to the ability to execute multiple processes or threads simultaneously, improving resource utilization and system efficiency. It allows several tasks to be in progress at the same time, either by running on separate processors or through context switching on a single processor.
Further reading / technical references:
Https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-intro.pdf
https://www.geeksforgeeks.org/operating-systems/concurrency-in-operating-system/

---

## Slide 7

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
One Core versus Two Cores
07

(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Execution resource — Slot 1 — Slot 2 — Slot 3 — Slot 4
One core — A — B — A — B
Core 0 — A — A — A — A
Core 1 — B — B — B — B
The first row shows interleaving. The last two rows show simultaneous execution. **Speaker Notes (Under Slide):**

This slide shows two separate scenarios for running tasks A and B. Each slot represents a time interval.
The rows are two separate scenarios. Slots are illustrative, equal-length intervals, not measured timing. A and B label tasks. Parallelism does not remove the need to synchronize shared data.

---

## Slide 8

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Process and Thread Concurrency
08
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CONCURRENT PROCESSES
CONCURRENT THREADS
Each process normally has its own virtual address space.
Processes coordinate through IPC.
Shared-memory mappings and external resources can still create shared state.
Threads in one process share its address space.
Each thread has its own registers and stack.
Shared variables make communication convenient and coordination necessary.
Separate processes can race too, for example when updating the same shared file. **Speaker Notes (Under Slide):**

This slide explains how processes and threads share data–and why both may need synchronization.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf

---

## Slide 9

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Shared Data and Thread-Local State
09
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

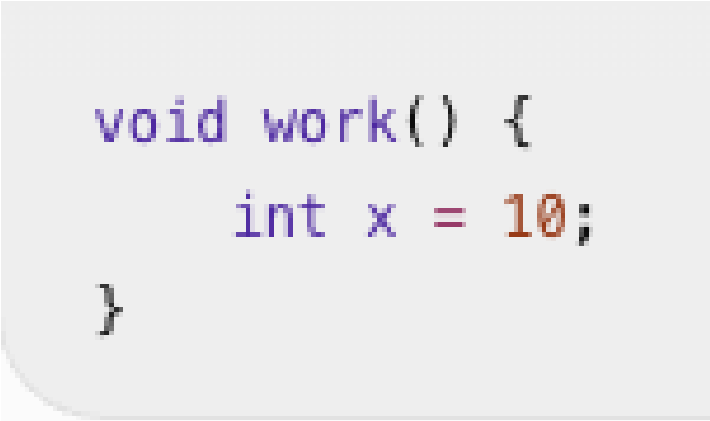
State — Usually shared? — Synchronization concern
Global variables and heap objects — Yes, within a process — Concurrent reads and writes
Thread registers — No — Execution context of that thread
Local stack variables — Usually private — A pointer can expose them to other threads
Files and shared-memory regions — May cross processes — A suitable cross-process protocol is needed
“Local variable” describes storage location, not a guarantee that no other thread can access it. **Speaker Notes (Under Slide):**
“Local variable” describes a storage location, not a guarantee that no other thread can access it.
This slide explains which data threads share and which execution information belongs to each thread. Here, state means the data and values a program currently holds.
Global variables and heap objectsThreads in the same process can access them. If multiple threads access the same data and at least one modifies it, coordination may be needed.
Thread registersEach thread has its own register values, such as its program counter and temporary calculation results. During a context switch, the system saves and restores these values
Local stack variablesEach function call normally has its own local variables on the calling thread’s stack. However, another thread can access them if given a valid pointer.

Files and shared-memory regionsThese can be accessed by different processes, so coordination must work across those processes–for example, using an appropriate file lock or shared semaphore.

---

## Slide 10

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Interleaved Execution
10
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
// Initial value: counter = 0
// Both threads execute:
r = counter;
r = r + 1;
counter = r;
One source-level increment may require several machine operations.
A context switch can occur between operations.
On multiple cores, operations can also overlap in real time.
The scheduler does not automatically make a multi-step update indivisible.




**Speaker Notes (Under Slide):**
Interleaved execution means that steps from different threads are mixed together. One thread can pause partway through an operation while another thread runs.
“”Example 1: r is a separate local register for each thread. Use a sequentially consistent teaching model with individually atomic loads and stores. Real unsynchronized C/C++ access to a non-atomic counter has undefined behavior.
“” Example 2: Each call has its own x. Changing Thread A’s x does not change Thread B’s x.
However, if Thread A gives Thread B the address of A’s x, then B can access that same variable while it remains alive.
A thread’s stack is not a protected memory compartment within the process.
Further reading / technical references:
https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-intro.pdf
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

---

## Slide 11

**Slide Content:**
Each thread has its own r, but both access the same counter. We can label their temporary values rA and rB.
Consider this possible execution order:
The final value is 1 instead of 2. Both threads calculated their result using the original value of 0. B’s write does not add another 1; it writes its already-calculated value of 1. This is a lost update caused by a race condition.
This can happen on one core through context switches, or on multiple cores with simultaneous execution.

| Step | Thread running | Operation | Shared `counter` |
|---|---|---|---|
| 1 | A | Reads `counter`: `rA = 0` | 0 |
| 2 | B | Reads `counter`: `rB = 0` | 0 |
| 3 | A | Calculates `rA = 0 + 1` | 0 |
| 4 | B | Calculates `rB = 0 + 1` | 0 |
| 5 | A | Writes `counter = rA` | 1 |
| 6 | B | Writes `counter = rB` | 1 |

```
r = counter;    // Read the shared value into a private temporary
r = r + 1;      // Increase the temporary value
counter = r;    // Write the result back to shared memory
```

**Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 12

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Nondeterministic Execution
11
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
WHY EXECUTION ORDER CHANGES
WHAT CAN REMAIN CORRECT
Timer interrupts and preemption
I/O completion and external events
Core scheduling, cache behavior, and competing work
Threads may finish in different orders.
A synchronized counter can still produce the same final total.
Correctness depends on required invariants, not one preferred schedule.
Nondeterminism is normal. An outcome that violates the specification is a bug. **Speaker Notes (Under Slide):**

Nondeterminism is normal. An outcome that violates the specification is a bug.
Nondeterministic execution means the order of execution can change between runs, even with the same program and input. Thread A might run first today, while Thread B runs first the next time.

---

## Slide 13

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Benefits and Challenges of Concurrency
12
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
BENEFITS
CHALLENGES
Keep an application responsive while work continues.
Overlap computation with I/O waits.
Use multiple cores when work can run in parallel.
Protect shared data and coordinate dependencies.
Avoid deadlock, starvation, and lost wakeups.
Control synchronization overhead and reproduce timing bugs.

More threads help only when useful overlap exceeds the added coordination cost. **Speaker Notes (Under Slide):**

This slide explains how concurrency helps applications–and what makes concurrent programs difficult to build correctly.
More threads help only when useful overlap exceeds the added coordination cost.
Responsiveness: An application can respond to the user while another task continues. For example, you can keep typing while a background task saves your document.
Overlap computation with I/O: While one task waits for a disk read or network response, the CPU can execute another task. This helps even on one core.
Use multiple cores: Independent tasks can execute simultaneously. For example, different cores can process different sections of an image.
The challenges are:
Protect shared data: Prevent conflicting updates, such as the lost increment in the counter example.
Coordinate dependencies: Ensure a task waits for something it needs–for example, a consumer must wait until a producer supplies data.
Deadlock: Tasks wait indefinitely for one another. For example, A holds a lock B needs, while B holds a lock A needs.
Starvation: One task keeps being denied CPU time or a resource while other tasks continue making progress.
Lost wakeup: A task misses a notification because checking a condition and going to sleep are not coordinated correctly. It may remain asleep even though it could proceed.
Synchronization overhead: Acquiring locks, waiting, and managing coordination consume time and resources.
Timing bugs: A bug may occur only under a particular execution order, making it difficult to reproduce and debug.

---

## Slide 14

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Concurrency on One Core - question
13
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
A single-core computer runs two threads that update the same counter.
1. Are the threads concurrent?
2. Are they executing in parallel?
3. Can a race condition occur?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
1-Are the threads concurrent? Yes.The CPU can alternate between them, allowing both to make progress during overlapping periods.
2-Are they executing in parallel? No.In this single-core model, only one thread executes at a time.
3-Can a race condition occur? Yes, if their updates are not properly synchronized.The CPU can switch threads in the middle of a counter update.

---

## Slide 15

**Slide Content:**
3.1 FOUNDATIONS OF CONCURRENCY
Concurrency on One Core - answer
14
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Yes. Their executions overlap in time through interleaving.
No. Only one thread executes on the core at an instant.
Yes. A context switch can split a read-modify-write update. **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 16

**Slide Content:**
Exercise :
Does adding a thread help?
A program runs on a single-core computer and performs two independent tasks:
Task A: Waits 10 milliseconds for a disk read.
Task B: Performs 6 milliseconds of CPU computation.
The disk can perform the read while the CPU executes Task B. Initially, ignore coordination overhead.
Questions:
If Task A finishes before Task B starts, how long do both tasks take?
If two threads overlap these tasks, what is the ideal total time?
Are these threads concurrent? Are they executing CPU instructions in parallel?
Suppose thread coordination adds 2 milliseconds to the ideal concurrent time. Does concurrency still improve performance? By how much?
If both tasks instead required only CPU computation, could one core execute them in parallel? What could two cores change? **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 17

**Slide Content:**
The useful overlap saves 6 ms before overhead. Since coordination costs only 2 ms, there is a net benefit.

**Answer key:**

| Question | Answer |
|---|---|
| 1 | **16 ms:** 10 ms waiting + 6 ms computation. |
| 2 | **10 ms:** B completes its computation during A's I/O wait. |
| 3 | **Concurrent: yes. CPU parallelism: no.** Their activities overlap, but the single core executes one thread's CPU instructions at a time. |
| 4 | **12 ms total:** 10 + 2. This saves **4 ms** compared with sequential execution. |
| 5 | On one core, their CPU instructions must take turns. Two cores could execute the independent computations simultaneously. |

**Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 18

**Slide Content:**
Exercise :
Shared counter, different outcomes
Two threads, A and B, belong to the same process. They share a global variable, initially counter = 10.
Each thread executes the following simplified pseudocode once, without synchronization. Each has its own private temporary variable r.
r = counter
r = r + 1
counter = r
Questions:
Which variable is shared, and which is private to each thread?
What final value is intended after both threads finish?
Show an execution order that produces an incorrect final value. Can this happen on one core?
Show an execution order that produces the correct value. Why might different runs produce different results?

If A and B belonged to separate processes, would ordinary global variables named counter automatically refer to the same data? How could the processes deliberately share data?
With a mutex protecting the entire increment, must A always finish before B? What final value should be guaranteed?
**Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 19

**Slide Content:**
Answer key:
counter is shared. A and B have separate temporary values, which we can call rA and rB.
The intended final value is 12.
One incorrect execution is:
The result is 11, a lost update. This can happen on one core through interleaving.

| Step | Thread A | Thread B | Shared counter |
|---|---|---|---|
| 1 | Reads 10 | — | 10 |
| 2 | — | Reads 10 | 10 |
| 3 | Calculates 11 | — | 10 |
| 4 | — | Calculates 11 | 10 |
| 5 | Writes 11 | — | 11 |
| 6 | — | Writes 11 | 11 |

**Speaker Notes (Under Slide):**
4. A completes all three steps, changing the counter to 11; B then reads 11 and changes it to 12. Scheduling and timing can change the execution order between runs.
5. No. Separate processes normally have separate address spaces. They can communicate through IPC, including deliberately shared memory, or coordinate access to a shared file.
6. Either thread may finish first. With correct locking, the final counter must be 12 after both finish.

---

## Slide 20

**Slide Content:**
4. A completes all three steps, changing the counter to 11; B then reads 11 and changes it to 12. Scheduling and timing can change the execution order between runs.
5. No. Separate processes normally have separate address spaces. They can communicate through IPC, including deliberately shared memory, or coordinate access to a shared file.
6. Either thread may finish first. With correct locking, the final counter must be 12 after both finish. **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 21

**Slide Content:**
Exam1, end of the session for October 7, and 8 **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 22

**Slide Content:**
SECTION 3.2

Race Conditions and Critical Sections
Protecting shared invariants under every allowed interleaving
3.2 / 8
15
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Protecting shared invariants under every allowed interleaving

---

## Slide 23

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
Race Conditions and Data Races
16
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
RACE CONDITION
DATA RACE
Correctness depends on an uncontrolled ordering of events.
Example: two threads both observe the final available seat and both book it.
A check-then-act sequence can race even when each separate operation is thread-safe.
Conflicting memory accesses occur without the required synchronization.
At least one access writes the same memory location.
In C/C++, a data race on ordinary shared data causes undefined behavior.
A data race is a memory-model violation. A race condition is the broader correctness problem. **Speaker Notes (Under Slide):**
For C/C++ the precise definition includes different threads, a non-atomic conflicting access, and absence of a happens-before ordering. Do not promise a finite list of outcomes for a real C/C++ program containing such a race.
Further reading / technical references:
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

---

## Slide 24

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
A Shared Increment Is a Multi-Step Operation
17
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
// Teaching model: counter starts at 0
// Each thread increments once
T1: a = counter;
a = a + 1;
counter = a;
T2: b = counter;
b = b + 1;
counter = b;
Intended final value: 2.
a and b are private to their threads.
The read and write are individually atomic in this model.
The entire increment is not atomic.
Two correct local calculations can produce an incorrect shared result. **Speaker Notes (Under Slide):**

This is a conceptual interleaving exercise, not valid race-free C code. C/C++ undefined behavior is discussed on the preceding slide.

---

## Slide 25

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
Lost Update: Execution Trace
18
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Step — Thread 1 — Thread 2 — counter
1 — a = counter // a = 0 — – — 0
2 — – — b = counter // b = 0 — 0
3 — a = a + 1 // a = 1 — – — 0
4 — – — b = b + 1 // b = 1 — 0
5 — counter = a — – — 1
6 — – — counter = b — 1
Thread 2 overwrites the same value. One logical increment disappears. **Speaker Notes (Under Slide):**

Ask students for a second legal interleaving that produces 2. Completing all of T1 before T2 is one example. These outcomes refer only to the stated abstract model.

---

## Slide 26

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
The Critical-Section Structure
19
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
while (true) {
entry_section();
critical_section();
exit_section();
remainder_section();
}
Entry: request permission to access protected state.
Critical: read or modify the shared invariant.
Exit: release the protection.
Remainder: perform unrelated work.
Critical sections using the same protected state must follow one consistent protocol. **Speaker Notes (Under Slide):**

Critical sections using the same protected state must follow one consistent protocol.

---

## Slide 27

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
Three Correctness Requirements
20
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Requirement — Meaning — Failure example
Mutual exclusion — At most one participant occupies the protected critical section. — Two threads update the same list links at once.
Progress — If entry is possible and threads want to enter, selection cannot be postponed indefinitely. — An uninterested thread controls whose turn it is.
Bounded waiting — After a request, there is a bound on how many times others may enter first. — One thread repeatedly loses the lock forever.
Mutual exclusion addresses safety. Progress and bounded waiting address liveness. **Speaker Notes (Under Slide):**

Standard assumptions: each participant continues to get execution opportunities, critical sections terminate, and a participant does not crash while holding the resource. Bounded waiting counts overtaking entries, not necessarily elapsed milliseconds.

---

## Slide 28

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
Progress and Bounded Waiting
21
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
PROGRESS
BOUNDED WAITING
Asks whether some waiting participant can advance.
A thread in its remainder section must not prevent entry.
Progress alone can allow one unlucky thread to starve.
Asks whether each request has a bounded amount of overtaking.
A fair queue can constrain who enters next.
A fixed waiting-time bound needs scheduling and execution-time assumptions too.
A system can keep doing work while one particular thread never gets served. **Speaker Notes (Under Slide):**

A system can keep doing work while one particular thread never gets served.

---

## Slide 29

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
Thread Safety and Reentrant Code
22
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
THREAD-SAFE CODE
REENTRANT CODE
Concurrent calls satisfy the documented contract.
Internal locking can protect shared mutable state.
Thread-safe individual methods do not automatically make a sequence atomic.
A new invocation can run before an earlier invocation finishes.
Per-call state avoids interference among overlapping invocations.
A nonrecursive internal mutex can deadlock if an interrupting call reenters the function.
Thread safety and reentrancy describe different usage guarantees. **Speaker Notes (Under Slide):**

Reentrancy can involve recursion, callbacks, or interruption. It is not synonymous with async-signal safety. POSIX signal handlers may call only the permitted async-signal-safe functions and must follow signal-specific data access rules.

---

## Slide 30

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
Shared Scratch State versus Per-Call State
23
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
SHARED SCRATCH BUFFER
CALLER-PROVIDED BUFFER
static char scratch[80];
char *format_id(int id) {
format_into(scratch, id);
return scratch;

}
void format_id(int id,
char *out) {
format_into(out, id);
}
// Each call uses its own out.
Separate buffers remove this shared-state hazard. Helper functions must satisfy the same contract. **Speaker Notes (Under Slide):**
Conceptual pseudocode. A real implementation must pass buffer capacity and check truncation. Caller-owned buffers help only if callers do not share the same mutable buffer unsafely. This example makes no signal-safety claim.

---

## Slide 31

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
One Remaining Seat - question
24
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
Two threads execute:
if (seats ¿ 0)
seats = seats - 1;
Each read and write is atomic. Is the reservation operation safe?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
Distinguish making a variable atomic from making the whole business operation atomic. The essential invariant is that successful reservations cannot exceed capacity.

---

## Slide 32

**Slide Content:**
3.2 RACE CONDITIONS AND CRITICAL SECTIONS
One Remaining Seat - answer
25
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
No. Both threads can read seats = 1 before either completes the update.
The check and update must form one protected operation.
Protect the entire reservation with one mutex, or use a suitable atomic compare-and-swap protocol. **Speaker Notes (Under Slide):**
Distinguish making a variable atomic from making the whole business operation atomic. The essential invariant is that successful reservations cannot exceed capacity.

---

## Slide 33

**Slide Content:**
SECTION 3.3
Software Synchronization Solutions
Evaluating protocols before relying on hardware primitives
3.3 / 8
26
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Evaluating protocols before relying on hardware primitives

---

## Slide 34

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Strict Alternation with a Turn Variable
27
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Pseudocode: atomic reads/writes and sequential consistency
// Shared: turn = 0
// Thread i, other thread j = 1 - i
while (true) {
while (turn != i) { }
critical_section();
turn = j;
remainder_section();
}
Only the thread named by turn can enter.
Each thread hands the next turn to the other.
This enforces alternation even when the other thread does not want to enter.
Under the teaching assumptions, strict alternation gives mutual exclusion but fails progress. **Speaker Notes (Under Slide):**
Assume atomic reads/writes and sequential consistency. This algorithm is not presented as valid plain-variable C/C++ synchronization.

---

## Slide 35

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Why Strict Alternation Fails Progress
28
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Step — Thread 0 — Thread 1 — turn
1 — Enters and exits the critical section — Remainder section — 1 after exit
2 — Wants to enter again — Still doing unrelated work — 1
3 — Spins although the critical section is empty — Does not request entry — 1
Thread 1 can delay Thread 0 while staying outside the entry protocol. **Speaker Notes (Under Slide):**

Thread 1 can delay Thread 0 while staying outside the entry protocol.

---

## Slide 36

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
A Shared Boolean Flag Is Not Enough
29
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
// Shared: locked = false
while (locked) { }
locked = true;
critical_section();
locked = false;
Thread 0 can observe locked == false.
Thread 1 can observe the same value before Thread 0 sets it.
Both then set locked = true and enter.
Testing and setting the flag must be one indivisible operation. **Speaker Notes (Under Slide):**

Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

---

## Slide 37

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Peterson's Two-Thread Solution
30
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
// Shared: flag[2] = {false, false}
// Shared: turn
// Thread i uses j = 1 - i
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { }
critical_section();
flag[i] = false;
flag[i] announces that i wants to enter.
turn = j gives the other thread priority when both want entry.
The loop waits only when the other thread is interested and has priority.
Teaching assumptions: two threads, atomic reads/writes, and sequential consistency. **Speaker Notes (Under Slide):**

Peterson combines interest with a tie-breaker. Do not implement this using ordinary C/C++ shared variables. Proper language atomics and memory ordering are required. The safest teaching translation uses sequentially consistent atomic variables throughout.

---

## Slide 38

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Peterson's Solution: Both Threads Request Entry
31
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Step — Thread 0 — Thread 1 — Result
1 — flag[0] = true — – — 0 is interested
2 — – — flag[1] = true — Both interested
3 — turn = 1 — – — 0 gives priority to 1
4 — – — turn = 0 — 1 gives priority to 0
5 — flag[1] && turn == 1 — flag[0] && turn == 0 — 0 enters, 1 waits
6 — Exits and clears flag[0] — Loop becomes false — 1 can enter
The last write to turn resolves simultaneous interest. **Speaker Notes (Under Slide):**

Keep both flags true while evaluating step 5. The trace picks one possible ordering of the doorway writes.

---

## Slide 39

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Peterson's Solution: Correctness Reasoning
32
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Property — Reason under the stated assumptions
Mutual exclusion — If both flags are true, turn gives priority to only one thread. A later entrant sees the other's interest

before passing.
Progress — If only one thread wants entry, the other flag is false. If both want entry, turn selects one.
Bounded waiting — After requesting entry, a thread can be overtaken at most once by the other thread before its next opportunity.
The proof depends on the memory model and on participants continuing to run. **Speaker Notes (Under Slide):**

Discuss why mutual-exclusion reasoning must include doorway interleavings, not only the case where both flags were already true. The overtaking bound is a bound on entries, not a real-time deadline.

---

## Slide 40

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Limitations of Software-Only Approaches
33
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
ASSUMPTIONS CAN FAIL
COSTS STILL REMAIN
Compilers can reorder or remove ordinary loads and stores.
Processors can expose memory operations in different orders.
C/C++ volatile does not provide interthread synchronization.
Busy waiting consumes CPU time.
Two-thread algorithms do not directly provide a general-purpose lock.
Production code usually uses tested synchronization libraries.
A correct abstract algorithm still needs a correct implementation for the language and hardware. **Speaker Notes (Under Slide):**
Further reading / technical references:
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf
https://docs.kernel.org/core-api/wrappers/memory-barriers.html

---

## Slide 41

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Evaluating the Three Software Attempts
34
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Protocol — Mutual exclusion — Progress — Bounded waiting
Separate test and set — Fails — No valid lock guarantee — No valid lock guarantee
Strict alternation — Yes — Fails when the other thread is idle — No complete entry guarantee
Peterson, two threads — Yes — Yes — Yes, under the stated assumptions
A protocol must satisfy all required properties, not just keep two threads out at once. **Speaker Notes (Under Slide):**

Strict alternation limits overtaking, but a requester can still wait forever for an uninterested participant. The table avoids treating that limited property as a satisfactory bounded-entry guarantee.

---

## Slide 42

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Peterson's Waiting Condition - question
35
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
Both flags are true and turn == 1.
1. Which thread may enter?

2. What lets the waiting thread proceed?
3. Does changing the flags to volatile make ordinary C code correct?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 43

**Slide Content:**
3.3 SOFTWARE SYNCHRONIZATION SOLUTIONS
Peterson's Waiting Condition - answer
36
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Thread 1 may enter. Thread 0 waits because flag[1] && turn == 1 is true.
Thread 1 clears flag[1] when it exits, so Thread 0's loop can become false.
No. C/C++ volatile does not replace atomic variables and the required memory ordering. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 44

**Slide Content:**
SECTION 3.4
Hardware Support for Synchronization
Atomic updates and memory ordering as building blocks
3.4 / 8
37
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Atomic updates and memory ordering as building blocks

---

## Slide 45

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Disabling Interrupts
38
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
LIMITED KERNEL USE
LIMITATIONS
Privileged kernel code can mask local interrupts briefly.
This can protect against certain interrupt-handler interleavings on that CPU.
The code must restore the previous interrupt state.
Other cores can continue accessing shared data.
Long interrupt-off intervals increase interrupt latency.
User programs cannot use this as a general lock, and protected code must not block.
Local interrupt masking alone does not provide mutual exclusion across multiple cores. **Speaker Notes (Under Slide):**
On a uniprocessor, the exact guarantee depends on the preemption and interrupt model. Nonmaskable events and other execution contexts require separate treatment. Kernels may combine a spinlock with local interrupt masking.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

---

## Slide 46

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Atomic Read-Modify-Write Operations
39
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
An atomic read-modify-write operation appears indivisible to competing accesses.
Hardware serializes competing atomic updates to the same location.
Test-and-Set, Compare-and-Swap, and exchange can implement lock protocols.
Atomicity and memory ordering are distinct properties. A lock needs both.
One atomic instruction protects one operation. A protocol protects a larger invariant. **Speaker Notes (Under Slide):**

Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf
https://docs.kernel.org/core-api/wrappers/memory-barriers.html

---

## Slide 47

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Test-and-Set
40
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
atomic test_and_set(L):
old = L
L = true
return old
// Acquire
while (test_and_set(L)) { }
critical_section()
// Release
store_release(L, false)
The entire primitive is atomic.
An old value of false means this caller acquired the lock.
An old value of true means another caller owns it.
Successful acquisition needs acquire ordering.
The separate test and set have become one indivisible operation. **Speaker Notes (Under Slide):**

Language-neutral pseudocode. L starts false. test_and_set must include appropriate acquire semantics for lock acquisition, and unlock uses release semantics. The simple loop provides no FIFO fairness guarantee.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

---

## Slide 48

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Compare-and-Swap (CAS)
41
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
atomic CAS(addr, expected, new):
old = *addr
if old == expected:
addr = new
return old
// L initially 0

while (CAS(&L, 0, 1) != 0) { }
critical_section()
store_release(L, 0)
CAS changes a value only if it matches the expected value.
This slide uses the convention “return the old value.”
Successful CAS acquires the lock when old == 0.
Real APIs may return a Boolean and update an expected-value argument.
Always check the return-value convention and memory-order contract of the actual API. **Speaker Notes (Under Slide):**
Pseudocode CAS must be atomic and have acquire semantics on successful lock acquisition. C/C++ compare_exchange differs from this old-value-return convention. Weak compare_exchange can fail spuriously.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

---

## Slide 49

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Atomic Swap / Exchange
42
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
// Shared: locked = false
// Local: key
key = true;
while (key) {
atomic_swap(locked, key);
}
critical_section();
store_release(locked, false);
atomic_swap exchanges the two values as one atomic operation.
If locked was false, key becomes false and this thread owns the lock.
If locked was true, key stays true and the loop continues.
Exchange-based APIs often return the previous shared value.
The same idea appears as an exchange instruction or an atomic exchange API. **Speaker Notes (Under Slide):**

Classical pseudocode has a shared lock and a thread-local key. Appropriate acquire semantics are required. Real hardware normally exchanges a register value with memory, rather than arbitrary pairs of memory locations.

---

## Slide 50

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Memory Barriers and Ordering
43
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
WHY ORDERING MATTERS
ACQUIRE AND RELEASE
A writer can update data and then announce that it is ready.
Without a synchronization relation, another thread cannot assume it observes those writes in that order.
A memory barrier enforces specified ordering constraints on memory accesses.
Release publishes preceding writes through a synchronization operation.
A matching acquire makes those writes visible to later reads.
Correct library locks supply the needed ordering for protected data.
A fence is not a substitute for mutual exclusion, and a compiler barrier alone is not a hardware fence. **Speaker Notes (Under Slide):**

Further reading / technical references:
https://docs.kernel.org/core-api/wrappers/memory-barriers.html
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

---

## Slide 51

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Publishing Data with Release and Acquire
44
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C++-style pseudocode
WRITER
READER
// One publication only
// ready is an atomic Boolean
// ready is initially false
data = 42;
ready.store(true, release);
while (!ready.load(acquire)) { }
// If the load observes true:
use(data);
The acquire that observes the release establishes visibility of data = 42. **Speaker Notes (Under Slide):**

data may be an ordinary variable because its one write happens-before the read through the ready atomic. There must be no later unsynchronized modification. A reusable publication protocol needs additional coordination. This is not a recommendation to spin for long waits.
Further reading / technical references:
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf
https://docs.kernel.org/core-api/wrappers/memory-barriers.html

---

## Slide 52

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Spinlocks
45
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
WHEN SPINNING CAN HELP
WHEN SPINNING HURTS
The expected wait is very short.
The owner can run on another core and release promptly.
Blocking and waking would cost more than brief spinning.
The owner is descheduled or cannot run.
The critical section performs slow I/O or blocking work.
Many threads repeatedly update one cache line.
Spinlock critical sections must stay short. Simple spinlocks do not guarantee bounded waiting. **Speaker Notes (Under Slide):**
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

---

## Slide 53

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Hardware Primitives versus Complete Locks
46

(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Mechanism — Provides — Does not automatically provide
Atomic Test-and-Set / exchange — Indivisible read and update — Fair queueing or low contention
Compare-and-Swap — Conditional atomic update — Correct multi-variable invariants
Acquire / release ordering — Required visibility ordering — Mutual exclusion by itself
Spinlock protocol — Exclusive access while held — Bounded waiting or efficient long waits
Correct lock design combines an atomic ownership protocol with ordering and a waiting policy. **Speaker Notes (Under Slide):**
Correct lock design combines an atomic ownership protocol with ordering and a waiting policy.

---

## Slide 54

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Two Test-and-Set Calls - question
47
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
The lock initially contains false. Two threads call atomic test_and_set on it.
1. Can both calls return false?
2. Can the winning thread be preempted?
3. Does the losing thread have a bounded waiting guarantee?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 55

**Slide Content:**
3.4 HARDWARE SUPPORT FOR SYNCHRONIZATION
Two Test-and-Set Calls - answer
48
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
No. Without an intervening unlock, only the first atomic call can observe false.
Yes. Atomic acquisition does not make the entire critical section immune to preemption.
No. This simple spinlock has no rule that limits how often others can overtake it. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 56

**Slide Content:**
SECTION 3.5
Mutexes and Semaphores
Exclusive ownership, resource permits, and blocking waits
3.5 / 8
49
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Exclusive ownership, resource permits, and blocking waits

---

## Slide 57

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES

Mutex Locks
50
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
mutex m;
lock(m);
// Shared invariant is protected
update_shared_state();
unlock(m);
Only one thread owns the mutex at a time.
The owner releases it after the protected operation.
All accesses that can conflict must obey the same locking rule.
Use cleanup paths or scope-based guards so errors do not leave the lock held.
A lock protects a rule about data. It does not automatically find every access to that data. **Speaker Notes (Under Slide):**
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

---

## Slide 58

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Pthreads: Protecting a Shared Counter
51
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C / POSIX Threads fragment
pthread_mutex_t m =
PTHREAD_MUTEX_INITIALIZER;
long counter = 0;
void *worker(void *arg) {
for (int k = 0; k ¡ 100000; ++k) {
pthread_mutex_lock(&m);
++counter;
pthread_mutex_unlock(&m);
}
return NULL;
}
Create two workers, then join both.
The expected final count is 200000.
The critical section includes the full increment.
Fragment omits headers, main, and error checks.
The final read occurs after both successful pthread_join calls. **Speaker Notes (Under Slide):**

Required header is pthread.h. Production code must inspect Pthreads return values. Each worker holds the mutex for each increment to make the teaching point; for throughput, a per-thread subtotal plus a final aggregation can be better. pthread_join synchronizes with thread completion.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

---

## Slide 59

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Mutex, Binary Semaphore, and Counting Semaphore
52
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Primitive — Initial state / range — Typical use — Ownership
Mutex — Unlocked or locked — Protect shared state — The locking thread owns it
Binary semaphore — 0 or 1 in a binary protocol — One permit or an event — No owner requirement
Counting semaphore — Nonnegative available permits — Pool of N equivalent resources — No owner requirement
A semaphore initialized to 1 can enforce exclusion, but its ownership rules differ from a mutex. **Speaker Notes (Under Slide):**
Some APIs expose only general counting semaphores. Initializing one to 1 does not prevent accidental extra posts from raising its count above 1. POSIX semaphores have an implementation-defined maximum count.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 60

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Semaphore wait() and signal()
53
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Abstract semaphore semantics
wait(S) / P / down
signal(S) / V / up
atomically:
if a permit is available:
consume one permit
return
else:
wait until a permit
can be acquired
atomically:
make one permit available
or hand it to a waiter
// A signal before a wait
// is remembered as a permit.
Each successful wait consumes one permit. Each signal returns or supplies one permit. **Speaker Notes (Under Slide):**
wait and signal are conventional textbook names, not all languages' literal API names. POSIX uses sem_wait and sem_post. Scheduling order depends on implementation and policy; do not assume FIFO.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 61

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Counting Semaphore: Three Resource Permits
54
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Event — Available permits — What happens
Initialize S = 3 — 3 — Three resources available
A calls wait(S) — 2 — A receives one resource
B calls wait(S), then C calls wait(S) — 0 — All resources in use
D calls wait(S) — 0 — D blocks
B returns a resource with signal(S) — 0 after handoff — D receives the released permit
The count tracks available permits, not the identity of the resource a thread receives. **Speaker Notes (Under Slide):**

This trace uses a nonnegative permit-count convention and direct handoff to a waiting thread. Some textbook implementations use a negative internal count to represent waiters. Those are alternative implementation conventions.

---

## Slide 62

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Blocking versus Busy Waiting
55
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
BUSY WAITING
BLOCKING
Repeatedly checks or retries while remaining runnable.
Consumes CPU time during the wait.
Can be useful for a very short delay.
Places the thread in a waiting state.
Allows the CPU to execute other runnable work.
Pays sleep, wakeup, and scheduling overhead.
Real mutex implementations may use a fast user-space path and block only after contention. **Speaker Notes (Under Slide):**
An adaptive mutex can spin briefly before sleeping. Waking makes a thread eligible to run; it does not guarantee immediate execution.
Further reading / technical references:
https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-locks.pdf

---

## Slide 63

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
A Blocking Semaphore Implementation
56
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Conceptual OS pseudocode
wait(S)
signal(S)
lock(S.internal)
if S.permits ¿ 0:
S.permits -= 1
unlock(S.internal)
else:
enqueue(S.queue, self)
park_and_unlock(S.internal)
// Resume with reserved permit
lock(S.internal)
if queue_empty(S.queue):
S.permits += 1
else:
t = dequeue(S.queue)
reserve_permit_for(t)
wake(t)
unlock(S.internal)
Queueing and sleeping must coordinate atomically with signaling to prevent a lost wakeup. **Speaker Notes (Under Slide):**
park_and_unlock is an internal primitive that prevents a wakeup from being lost between enqueue and sleep. It releases the internal lock and returns only after this thread receives its reserved permit. Implementations may use wake tokens or

an equivalent mechanism. Ordinary unlock followed by sleep is insufficient. This model does not assume application-level spurious returns.

---

## Slide 64

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
A Semaphore Can Enforce Event Ordering
57
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
THREAD A
THREAD B
// done initially 0
prepare_result();
signal(done);
wait(done);
consume_result();
Thread B continues only after a permit records completion of prepare_result(). **Speaker Notes (Under Slide):**

The synchronization primitive must have the required publication/visibility semantics. This is a one-event example. With repeated production, one must additionally coordinate buffer ownership and prevent overwrite.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 65

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Common Semaphore Errors
58
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Mistake — Possible result — Repair
Wrong initial count — Excess access or no progress — Set the count to actual permits
Missing signal — A permit leaks — Release on every exit after acquisition
Extra signal — Capacity can be exceeded — Balance acquisitions and releases
Wait while holding a needed mutex — Circular waiting — Wait for availability before taking that mutex
Assuming FIFO wakeups — Starvation may occur — Use an explicit fairness policy
Correct initialization and operation ordering are part of the algorithm. **Speaker Notes (Under Slide):**

Correct initialization and operation ordering are part of the algorithm.

---

## Slide 66

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Semaphore Initialization - question
59
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
Choose the initial value for each semaphore:
1. Mutual exclusion for one critical section
2. A pool with four available connections
3. “Wait until initialization has finished”
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 67

**Slide Content:**
3.5 MUTEXES AND SEMAPHORES
Semaphore Initialization - answer
60
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
1 permit for mutual exclusion, with balanced wait and signal operations.
4 permits for four available connections.
0 permits for the completion event. The initializer signals after publishing the result. **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 68

**Slide Content:**
SECTION 3.6
Monitors and Condition Variables
Keeping shared state, its lock, and its waiting rules together
3.6 / 8
61
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Keeping shared state, its lock, and its waiting rules together

---

## Slide 69

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
The Monitor Abstraction
62
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
A monitor groups shared state with the operations that access it.
Only one thread executes protected monitor code at a time.
Condition variables let a thread wait for a state predicate.
A language runtime or a library pattern manages entry and exit locking.
Encapsulation makes the synchronization rule easier to apply consistently. **Speaker Notes (Under Slide):**

A condition predicate is a Boolean statement about shared state, such as count ¿ 0. The condition variable itself does not contain that Boolean state. Java synchronized methods are one monitor-style mechanism; Pthreads can implement the pattern using a mutex and condition variables.
Further reading / technical references:
https://pages.cs.wisc.edu/˜bart/537/lecturenotes/s8.html

---

## Slide 70

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Waiting for a Condition
63
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
lock(m);
while (!predicate()) {
wait(cv, m);
}

use_protected_state();
unlock(m);
The caller holds m while checking the predicate.
wait atomically releases m and begins waiting.
Before wait returns, the thread reacquires m.
The loop checks whether the condition is still true.
The state predicate and the wait must use the same synchronization discipline. **Speaker Notes (Under Slide):**

POSIX example: pthread_cond_wait(&cv, &m). A real implementation must check errors. Keep the predicate protected by the associated mutex. Returned wait does not imply that the predicate is true.
Further reading / technical references:
https://pubs.opengroup.org/onlinepubs/9799919799/functions/pthread_cond_clockwait.html
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf

---

## Slide 71

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Condition-Variable Operations
64
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Operation — Effect — What to remember
wait(cv, m) — Releases m, waits, then reacquires m before returning — The caller must check the predicate again
signal(cv) — Wakes one eligible waiter, if one exists — The signal is not stored for a future waiter
broadcast(cv) — Wakes all current waiters — They compete to reacquire the mutex
For a simple reliable pattern, change the predicate and signal while holding its mutex. **Speaker Notes (Under Slide):**

The exact number awakened by POSIX signal can occasionally exceed one because of implementation effects. The practical contract is to avoid relying on exactly one successful consumer. A broadcast does not let all threads hold the mutex simultaneously.
Further reading / technical references:
https://pubs.opengroup.org/onlinepubs/9799919799/functions/pthread_cond_clockwait.html

---

## Slide 72

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Mesa versus Hoare Monitor Behavior
65
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Question — Mesa-style behavior — Hoare-style behavior
After signal — Signaler continues in the monitor — Control transfers immediately to a waiter
Signaled thread — Becomes eligible and later reacquires the lock — Runs with the monitor lock immediately
Predicate at resumption — May have changed before the waiter runs — Handoff preserves the signaled condition at that point
Usual programming rule — Recheck the predicate in a while loop — Reason from the stronger handoff semantics
Common condition-variable APIs use Mesa-like behavior. A wakeup is permission to recheck. **Speaker Notes (Under Slide):**
Pure Hoare monitor semantics park the signaler while the awakened waiter runs, then later allow the signaler to continue. Modern APIs may also allow spurious wakeups. While loops remain the safe default for practical Mesa-style condition-variable programming.
Further reading / technical references:
https://pages.cs.wisc.edu/~bart/537/lecturenotes/s8.html
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf

---

## Slide 73

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Why a while Loop Is Necessary
66
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Step — Action — Buffer count
1 — Consumer C1 waits for count ¿ 0 — 0
2 — Producer adds an item and signals C1 — 1
3 — Another consumer C2 acquires the mutex first and removes it — 0
4 — C1 reacquires the mutex and rechecks count ¿ 0 — 0
5 — C1 waits again instead of removing from an empty buffer — 0
Even without a spurious wakeup, another thread can consume the condition first. **Speaker Notes (Under Slide):**

Assume Mesa behavior and no fairness restriction that prevents C2 from acquiring first. The while loop addresses both changed state and spurious wakeups.
Further reading / technical references:
https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-cv.pdf

---

## Slide 74

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Condition Variable versus Semaphore
67
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Aspect — Condition variable — Semaphore
Persistent state — Predicate lives in shared data — Permit count lives in the semaphore
Signal with no waiter — No stored notification — Increases the available permit count
Waiting and a mutex — Atomically releases, then reacquires it — Does not release an unrelated mutex
Typical purpose — Wait for a protected state condition — Count resources or record events
The similar operation names hide different semantics. **Speaker Notes (Under Slide):**

Further reading / technical references:
https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-cv.pdf
https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-sema.pdf

---

## Slide 75

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Lost Notification or Safe Wait? - question
68
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
The producer sets ready = true while holding m and signals cv before the consumer arrives.
The consumer later locks m and executes:
while (!ready) wait(cv, m);
Will it wait forever because it missed the signal?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 76

**Slide Content:**
3.6 MONITORS AND CONDITION VARIABLES
Lost Notification or Safe Wait? - answer
69
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
No. It observes ready == true under the mutex and skips wait.
The shared predicate remembers that the work is ready.
A consumer that waits unconditionally could miss the earlier signal and block indefinitely. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 77

**Slide Content:**
SECTION 3.7
Classical Synchronization Problems
Applying safety and liveness requirements to complete protocols
3.7 / 8
70
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**

Applying safety and liveness requirements to complete protocols

---

## Slide 78

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Classical Problems and Their Constraints
71
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Problem — Shared resource — Main coordination question
Producer-Consumer — Buffer of items — When may an item be inserted or removed?
Readers-Writers — Shared data set — Which accesses may overlap safely?
Dining Philosophers — Pairs of neighboring utensils — How can multiple-resource acquisition progress?
Sleeping Barber — One server and a finite waiting room — How do arrivals, capacity, and service interact?
For each problem, define the invariant before choosing the primitive. **Speaker Notes (Under Slide):**

For each problem, define the invariant before choosing the primitive.

---

## Slide 79

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
The Bounded-Buffer Problem
72
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Producers insert items. Consumers remove items.
A buffer of capacity N may contain between 0 and N items.
A producer must wait when the buffer is full.
A consumer must wait when the buffer is empty.
Only one thread at a time updates shared buffer bookkeeping.
Producer-Consumer describes the roles. A bounded buffer adds a finite capacity. **Speaker Notes (Under Slide):**

Useful invariants: 0 ¡= count ¡= N, each produced item is removed at most once, and queue head/tail remain valid. Assume N ¿ 0 and operations eventually finish.

Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 80

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Bounded Buffer with Semaphores
73
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
PRODUCER
CONSUMER
item = produce()
wait(empty)
wait(mutex)
insert(item)
signal(mutex)
signal(full)
wait(full)
wait(mutex)
item = remove()
signal(mutex)
signal(empty)
consume(item)
Initialize empty = N, full = 0, and mutex = 1. Wait for availability before taking mutex. **Speaker Notes (Under Slide):**
Produce and consume execute outside the buffer critical section. All operations are assumed to complete successfully. A real program must handle cancellation and errors without leaking permits. mutex is a binary semaphore in this classical version, although a mutex lock can replace it.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 81

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Bounded Buffer: Capacity Two
74
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Event — empty — full — Buffer
Initial state — 2 — 0 — [ ]
Producer completes insertion of A — 1 — 1 — [A]
Producer completes insertion of B — 0 — 2 — [A, B]
Producer blocks while reserving space for C — 0 — 2 — [A, B]
Consumer completes removal of A — 0 after handoff — 1 — [B]
Producer completes insertion of C — 0 — 2 — [B, C]
A permit may be reserved before the corresponding buffer operation finishes. **Speaker Notes (Under Slide):**

After the consumer signals empty, its space permit transfers to the blocked producer, so the available empty count remains 0 in the shown handoff. In that interval empty + full need not equal N. At stable boundaries without reservations, it does. Counts alone cannot describe all in-flight operations.

---

## Slide 82

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
A Bounded-Buffer Ordering Bug
75
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
// Incorrect consumer order
wait(mutex);
wait(full);
item = remove();
signal(mutex);
signal(empty);
Assume the buffer is empty.
The consumer holds mutex and blocks on full.
A producer needs mutex to insert an item and signal full.
Neither can complete the step needed by the other.
Repair: wait(full) before wait(mutex). Detailed deadlock analysis follows in Chapter 4. **Speaker Notes (Under Slide):**
Repair: wait(full) before wait(mutex). Detailed deadlock analysis follows in Chapter 4.

---

## Slide 83

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Bounded Buffer with Monitor Conditions
76
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
put(item)
get()
lock(m)
while count == N:
wait(not_full, m)
insert(item)
count += 1
signal(not_empty)
unlock(m)
lock(m)
while count == 0:
wait(not_empty, m)
item = remove()
count -= 1
signal(not_full)
unlock(m)
return item
One mutex protects count and the buffer. Two conditions describe different reasons to wait. **Speaker Notes (Under Slide):**
Pseudocode defines insert and remove as buffer index/data operations that do not themselves modify count. This avoids double-counting. Both functions reacquire m after every wait and reevaluate their predicates. Assumes homogeneous producer/consumer wait predicates.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf

---

## Slide 84

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
The Readers-Writers Problem
77
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
ACCESS RULES
POLICY CHOICES
Multiple readers may access the data at the same time.
A writer requires exclusive access.
No reader may overlap a writer on the protected data.
Reader preference can delay writers indefinitely.
Writer preference can delay readers indefinitely.
A fair admission policy controls overtaking for both groups.
An exclusion rule alone does not define who gets served next. **Speaker Notes (Under Slide):**

Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 85

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Readers-Writers: Reader-Preference Protocol
78
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
READER
WRITER
wait(rmutex)
readers += 1
if readers == 1: wait(room)
signal(rmutex)
read_data()
wait(rmutex)
readers -= 1
if readers == 0: signal(room)
signal(rmutex)
wait(room)
write_data()
signal(room)
// Initial values
readers = 0
rmutex = 1
room = 1
The first reader acquires room. The last reader releases it. **Speaker Notes (Under Slide):**

room is a semaphore because the first and last readers can be different threads. Replacing it with an owner-based mutex would be incorrect. rmutex protects readers. This solution permits writer starvation. It deliberately waits(room) while holding rmutex so that later readers cannot bypass the first reader before room is acquired; writers do not need rmutex.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

---

## Slide 86

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Readers-Writers Fairness

79
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Under reader preference, new readers can join while a writer waits.
An endless stream of readers can keep readers ¿ 0 indefinitely.
A turnstile or ordered admission queue can prevent newcomers from bypassing a waiting writer.
Starvation freedom still depends on the admission primitive and scheduling assumptions.
A readers-writers lock is useful only when its policy and overhead match the workload. **Speaker Notes (Under Slide):**

Do not claim every semaphore-based turnstile guarantees fairness. FIFO or another explicit bounded-overtaking policy is needed for a bounded-waiting argument. Reader-heavy does not automatically mean a readers-writers lock outperforms a mutex.

---

## Slide 87

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
The Dining Philosophers Problem
80
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Naive C-like pseudocode for philosopher i
while (true) {
think();
wait(F[i]);
wait(F[(i + 1) % 5]);
eat();
signal(F[(i + 1) % 5]);
signal(F[i]);
}
Five philosophers alternate between thinking and eating.
Five utensils sit between neighboring philosophers.
Each philosopher needs both adjacent utensils to eat.
Each utensil is a semaphore initialized to 1.
If everyone picks up one utensil first, all five can become stuck waiting for the second. **Speaker Notes (Under Slide):**

Philosopher Pi requires Fi and F(i+1 mod 5). This intentionally naive protocol demonstrates a deadlock risk. The next slide traces the all-hold-one state.
Further reading: https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-sema.pdf

---

## Slide 88

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Dining Philosophers: A Deadlock Trace
81
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Philosopher — Holds — Waits for
P0 — F0 — F1
P1 — F1 — F2
P2 — F2 — F3
P3 — F3 — F4
P4 — F4 — F0
Each philosopher holds one utensil and waits for the next. No one can eat or release. **Speaker Notes (Under Slide):**

Five philosophers sit around a table. Philosopher Pi requires Fi and F(i+1 mod 5). Each utensil has one permit. The trace assumes all pick up Fi first and keep it while waiting for the second. This table records the exact resource dependency cycle.
Further reading / technical references:
https://pages.cs.wisc.edu/˜remzi/OSTEP/threads-sema.pdf

## Slide 89

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Dining Philosophers: Ways to Avoid This Deadlock
82
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Approach — Rule — Remaining concern
Global resource order — Always acquire the lower-numbered utensil first — Fair access is still needed
Limit admission — At most four philosophers try to acquire utensils at once — The admission semaphore may be unfair
Central waiter / monitor — Grant both utensils together when available — The waiter needs a starvation policy
Avoiding deadlock does not automatically prevent starvation. **Speaker Notes (Under Slide):**

Assume each admitted philosopher continues to execute and releases both utensils after a finite meal. N-1 admission for N philosophers breaks the all-hold-one circular configuration. Full prevention/avoidance theory is intentionally left for Chapter 4.

---

## Slide 90

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
The Sleeping Barber Problem
83
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
One barber serves one customer at a time.
The shop has W waiting chairs in addition to the service chair.
An arriving customer joins the queue if a waiting chair is available, otherwise leaves.
The barber sleeps when the queue is empty and wakes when work arrives.
A shared queue and per-customer handshakes prevent lost or mismatched service.
Specify capacity, service order, and completion notification separately. **Speaker Notes (Under Slide):**

This teaching model assumes W ¿= 1 and every customer first joins the waiting queue. Dequeueing frees a waiting chair. A direct-to-service-chair optimization can be added but is not needed for the synchronization lesson.

---

## Slide 91

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Sleeping Barber: Customer Protocol
84
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Conceptual pseudocode, W $\geq$ 1
lock(m)
if queue.size == W:
unlock(m)
leave()
return
enqueue(queue, c)
unlock(m)
signal(customers)
wait(c.called)
sit_in_service_chair()
signal(c.seated)
wait(c.done)
leave_service_chair()

signal(c.left)
Join the FIFO queue only when a waiting chair is available.
Each customer has private handshake semaphores.
Wait for service completion, then confirm that the chair is free.
customers and each customer's called, seated, done, and left semaphores start at 0. **Speaker Notes (Under Slide):**

m starts unlocked. The queue is FIFO, and each customer owns four distinct semaphores. Assume customers remain until service completion, semaphores do not spuriously return, and operations succeed. A token cannot be consumed by the wrong customer because the service handshakes are per customer. Cancellation, shop closure, and abandoned customers need additional protocol. The left handshake ensures that the current customer vacates the service chair before the barber admits the next customer.

---

## Slide 92

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Sleeping Barber: Barber Protocol
85
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Conceptual pseudocode
while true:
wait(customers)
lock(m)
c = dequeue(queue)
unlock(m)
signal(c.called)
wait(c.seated)
cut_hair(c)
signal(c.done)
wait(c.left)
Wait for an arrival token before accessing the queue.
Remove one customer while holding the queue mutex.
Wait for that customer to sit before starting service.
Wait for the service chair to become free before admitting the next customer.
A FIFO queue plus per-customer handshakes keeps service in order. **Speaker Notes (Under Slide):**

m starts unlocked. The queue is FIFO, and each customer owns four distinct semaphores. Assume customers remain until service completion, semaphores do not spuriously return, and operations succeed. A token cannot be consumed by the wrong customer because the service handshakes are per customer. Cancellation, shop closure, and abandoned customers need additional protocol. The left handshake ensures that the current customer vacates the service chair before the barber admits the next customer.

---

## Slide 93

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Correctness and Fairness Checklist
86
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Question — What a solution must establish
Safety — Capacity, exclusive access, and data invariants always hold
Progress — Enabled work can eventually proceed
Starvation freedom — Every admitted request eventually receives service
Bounded waiting — A defined bound limits overtaking
Assumptions — Scheduling, finite operations, cancellation, and failures are explicit
Test traces can reveal a flaw. A correctness argument must cover all allowed executions. **Speaker Notes (Under Slide):**

Test traces can reveal a flaw. A correctness argument must cover all allowed executions.

---

## Slide 94

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Bounded Buffer: Find the Bug - question
87
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
Capacity N = 4. A proposed solution initializes:
empty = 4, full = 1, mutex = 1
The actual buffer is empty. What can go wrong before any producer runs?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 95

**Slide Content:**
3.7 CLASSICAL SYNCHRONIZATION PROBLEMS
Bounded Buffer: Find the Bug - answer
88
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
A consumer can successfully wait(full) even though no item exists.
The consumer can enter the buffer operation and attempt to remove nonexistent data.
Initialize full = 0. Semaphore counts must represent actual resources or events. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 96

**Slide Content:**
SECTION 3.8
Practical Concurrent Programming
Choosing synchronization rules that remain correct and scale
3.8 / 8
89
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026 **Speaker Notes (Under Slide):**
Choosing synchronization rules that remain correct and scale

---

## Slide 97

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Thread-Safe Data Structures
90
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
A DOCUMENTED CONTRACT
COMPOUND OPERATIONS
Define which methods may run concurrently.
Protect compound invariants such as head, tail, and count together.
Document iteration behavior and object lifetime.
if (!queue.empty()) queue.pop() contains a gap.
Another thread can remove the last item during that gap.
Prefer one try_pop operation that checks and removes atomically.

Thread-safe methods do not make arbitrary combinations of those methods atomic. **Speaker Notes (Under Slide):**

Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks-usage.pdf

---

## Slide 98

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Consistent Lock Ordering
91
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
INCONSISTENT ORDER
ONE GLOBAL ORDER
// Thread A
lock(account_1)
lock(account_2)
// Thread B
lock(account_2)
lock(account_1)
// Both threads
first = lower_unique_id(a, b)
second = higher_unique_id(a, b)
lock(first)
lock(second)
transfer()
unlock(second)
unlock(first)
Use the same stable order on every path that acquires these locks. **Speaker Notes (Under Slide):**

This pseudocode assumes distinct account objects with unique stable IDs and a shared locking policy. A transfer to the same account needs special handling so the same nonrecursive mutex is not acquired twice. Lock ordering prevents cycles among these locks but does not by itself guarantee fairness or eliminate unrelated deadlocks.

---

## Slide 99

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Lock Granularity
92
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Choice — Benefit — Cost / risk
Coarse-grained: one lock for a structure — Simple invariant and easier reasoning — Unrelated operations serialize
Fine-grained: a lock per bucket or region — Independent operations can overlap — More locks, ordering rules, and lifetime complexity
Partitioned ownership: each worker owns its state — Less shared mutable state — Coordination is needed when data crosses partitions
Begin with a correct design. Increase granularity only where measurements show contention. **Speaker Notes (Under Slide):**
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks-usage.pdf

---

## Slide 100

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Atomic Variables
93
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C++-style pseudocode
// C++-style pseudocode
atomic¡long¿ count = 0;
// Each worker
count.fetch_add(1, relaxed);
// After joining all workers
print(count.load());
fetch_add makes this increment indivisible.
Relaxed ordering can suffice for an independent count.
The joins establish worker completion before the final read.
An atomic count does not protect a linked list, queue, or multi-field invariant.
Choose the weakest ordering only when you can justify every required dependency. **Speaker Notes (Under Slide):**

The example counts independent events and does not publish associated payload data. Use a range that cannot overflow. Atomics are not necessarily lock-free for every type/platform. Lock-free algorithms also need safe memory reclamation and correct ABA handling where applicable.
Further reading / technical references:
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

---

## Slide 101

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Thread Pools and Bounded Concurrency
94
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
A pool reuses workers and takes tasks from a synchronized queue.
A bounded queue requires a policy when full: wait, reject, or apply backpressure.
CPU-bound and I/O-bound tasks have different worker-count needs.
Shutdown must define whether queued work drains or is canceled.
Workers that wait for tasks queued to the same saturated pool can deadlock.
A thread pool manages concurrency. It does not remove races inside the tasks. **Speaker Notes (Under Slide):**

Connect to Chapter 2 thread-pool fundamentals. This slide adds queue synchronization, lifecycle, and dependency hazards. Avoid a universal “one thread per core” rule for all workloads.

---

## Slide 102

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Debugging Races with ThreadSanitizer
95
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Example command on a supported Clang platform
clang -std=c11 -O1 -g \
-fsanitize=thread -pthread \
counter.c -o counter_tsan
./counter_tsan
ThreadSanitizer instruments a program to detect data races during execution.
Use debug symbols to locate the conflicting accesses.
Read both access stacks and their synchronization context.
A clean run does not prove that every possible execution is race-free.

Dynamic race detectors find executed problems. They cannot replace invariant-based reasoning. **Speaker Notes (Under Slide):**
Use on a supported platform and compiler installation. Not every logical race, deadlock, or starvation bug is a data race. Tool overhead changes timing. Official instructions recommend -g and -O1 or higher.
Further reading / technical references:
https://clang.llvm.org/docs/ThreadSanitizer.html

---

## Slide 103

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
A Practical Debugging Workflow
96
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Write the invariant and identify every access to the shared state.
Reduce the failure to a small reproducible case.
Use barriers or explicit handshakes to exercise a suspicious interleaving.
Run a race detector and inspect lock order and condition predicates.
Retest with different thread counts and workloads after repairing the protocol.
sleep() may expose a bug during a test, but it does not repair synchronization. **Speaker Notes (Under Slide):**

Logging and debugger breakpoints perturb timing. Preserve enough evidence to identify the conflicting accesses. If a failure disappears after adding a printf, it may still be present.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-bugs.pdf
https://clang.llvm.org/docs/ThreadSanitizer.html

---

## Slide 104

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Performance and Scalability Trade-offs
97
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Issue — Why performance drops — Possible response
Lock contention — Many threads serialize on one owner — Shorten the protected operation or partition state
False sharing — Independent writable data share a cache line — Separate hot per-thread data when justified
Blocking overhead — Frequent sleep/wakeup transitions cost time — Batch useful work or reconsider granularity
Excess workers — Scheduling and cache costs grow — Bound concurrency and measure throughput
Correctness comes first. Then measure the actual workload before changing the design. **Speaker Notes (Under Slide):**
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks-usage.pdf

---

## Slide 105

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Amdahl's Law: Synchronization Limits Speedup
98
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Let s be the serial fraction and p the number of processors.
Ideal speedup is at most 1 / (s + (1 - s) / p).
If s = 0.20 and p = 4, the ideal speedup is 1 / 0.40 = 2.5.
As p grows without bound, this example approaches a maximum speedup of 5.

Contention, cache traffic, and scheduling overhead can reduce real speedup further.
Adding cores cannot eliminate the part of the workload that must remain serial. **Speaker Notes (Under Slide):**

Illustrative calculation, not a measured benchmark. Assumes fixed work, a constant serial fraction, and ideal parallel execution of the remainder. The serial fraction includes inherently serial work and may include serialized critical sections. Real scaling can change the effective fraction.

---

## Slide 106

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Choosing a Synchronization Mechanism
99
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
TABLE

Need — Common starting point — Key question
Protect a shared invariant — Mutex / monitor — Do all conflicting accesses follow it?
Limit concurrent resource users — Counting semaphore — Does each permit represent a resource?
Wait for a state predicate — Condition variable with mutex — Is the predicate checked in a while loop?
Update one independent value — Atomic operation — What ordering is actually required?
Very short kernel-level wait — Spinlock where appropriate — Can the owner run and release promptly?
Choose based on the required guarantee, then consider cost and fairness. **Speaker Notes (Under Slide):**

Choose based on the required guarantee, then consider cost and fairness.

---

## Slide 107

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Code Review Activity: A Flawed Queue
100
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
lock(m);
if (queue.empty()) {
wait(not_empty, m);
}
item = queue.pop();
unlock(m);
Find an interleaving that makes pop unsafe.
Identify the predicate and its protecting mutex.
Repair the code and explain the change.
If using AI, keep the prompt log and verify every proposed fix.
A plausible explanation is not evidence that a concurrent program is correct. **Speaker Notes (Under Slide):**

Activity aligned with the AI review-and-debug approach in the supplied course plan. Students should provide a concrete counterexample and a correctness explanation, not just a code edit.

---

## Slide 108

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Code Review Activity: Corrected Queue
101
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
C-like pseudocode
lock(m);
while (queue.empty()) {

wait(not_empty, m);
}
item = queue.pop();
unlock(m);
Another consumer may take the item before this waiter reacquires m.
A spurious wakeup may occur even without a new item.
The while loop rechecks queue.empty() under the mutex.
Producers must use the same mutex and signal after adding work.
The predicate loop preserves the invariant across every return from wait. **Speaker Notes (Under Slide):**

In a bounded queue, pop should also signal not_full. This review example uses an unbounded work queue, so only not_empty appears.
Further reading / technical references:
https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf

---

## Slide 109

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Chapter Review - question
102
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
CLASSROOM DISCUSSION
1. Can mutual exclusion hold while a thread starves?
2. Does a condition signal store a future permit?
3. Does an atomic variable make a multi-step transaction atomic?
4. Does deadlock freedom imply fairness?
Think individually, compare with a partner, and justify your answer using an execution trace. **Speaker Notes (Under Slide):**
(No additional speaker notes)

---

## Slide 110

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Chapter Review - answer
103
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
1. Yes. Safety alone does not guarantee that every thread eventually enters.
2. No. A condition signal has no stored permit. The shared predicate records state.
3. No. The entire invariant needs an appropriate atomic operation or protocol.
4. No. A system can avoid deadlock yet repeatedly bypass one request. **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 111

**Slide Content:**
3.8 PRACTICAL CONCURRENT PROGRAMMING
Chapter 3 Summary
104
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Interleaving and parallelism can expose shared-state bugs.
A correct protocol needs safety and the required liveness guarantees.
Locks protect invariants, semaphores count permits, and conditions wait for predicates.
Memory ordering is part of real synchronization correctness.
Next: Chapter 4 examines deadlock conditions, prevention, avoidance, detection, and recovery.

The central question: what must stay true under every allowed execution? **Speaker Notes (Under Slide):**

The central question: what must stay true under every allowed execution?

---

## Slide 112

**Slide Content:**
CHAPTER 3 REFERENCES
Further Reading: Concurrency Concepts
105
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
Operating Systems: Three Easy Pieces
Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau
Ch. 26-32: concurrency, thread APIs, locks, concurrent data structures, condition variables, semaphores, and concurrency bugs.
https://pages.cs.wisc.edu/˜remzi/OSTEP/
Monitor semantics
University of Wisconsin CS 537 notes, Section 8: Monitors.
https://pages.cs.wisc.edu/˜bart/537/lecturenotes/s8.html **Speaker Notes (Under Slide):**

Examples and traces in this deck are original teaching examples and adaptations of standard synchronization algorithms. Individual slide notes identify relevant technical references. Supplied Chapter 1 and Chapter 2 decks establish the course context and visual style.

---

## Slide 113

**Slide Content:**
CHAPTER 3 REFERENCES
Further Reading: APIs and Memory Ordering
106
(c) Dr. Roya Hosseini — CSC4320, CSC6320 — GSU — Fall 2026
POSIX condition variables
The Open Group Base Specifications, Issue 8.
https://pubs.opengroup.org/onlinepubs/9799919799/functions/pthread_cond_clockwait.html
C11 memory model and atomics
WG14 N1570, especially 5.1.2.4 and 7.17.
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf
Linux memory barriers and Clang ThreadSanitizer
Kernel documentation and race-detector documentation.
https://docs.kernel.org/core-api/wrappers/memory-barriers.html
https://clang.llvm.org/docs/ThreadSanitizer.html **Speaker Notes (Under Slide):**

(No additional speaker notes)

---

## Slide 114

**Slide Content:**
End of Chapter 3 **Speaker Notes (Under Slide):**

(No additional speaker notes)

---